CS228: Introduction to Deep Learning

Lecture 5: gradient descent

Instructor: Zhouxing Shi

The perceptron algorithm

- Initialize W (random or simply 0)
- For each training example, **update W only if it's misclassified**
  - Positive sample misclassified:

    $$\mathbf{W} \leftarrow \mathbf{W} + X_i$$

  - Negative sample misclassified:

    $$\mathbf{W} \leftarrow \mathbf{W} - X_i$$

Generally:

$$\mathbf{W} \leftarrow \mathbf{W} + y_i X_i$$

$$(y_i \in \{-1, 1\})$$

Whiteboard: demo

Convergence

As long as the classes are **linearly separable**:

The perceptron algorithm is **guaranteed to converge to perfect classification** within a number of misclassifications

Convergence

Bounded scale:

$$\|X_i\| \leq R$$

$$\|W^*\| = 1$$

- Linearly separable:

$$\forall 1 \leq i \leq n, \;\; y_i(\mathbf{W}^* \cdot X_i) > \gamma \; (\gamma > 0)$$

- Maximum number of misclassifications: $\left(\frac{R}{\gamma}\right)^2$

- If not linearly separable, the algorithm does not converge
- Cannot be applied to learn MLPs

Whiteboard: proof (optional)

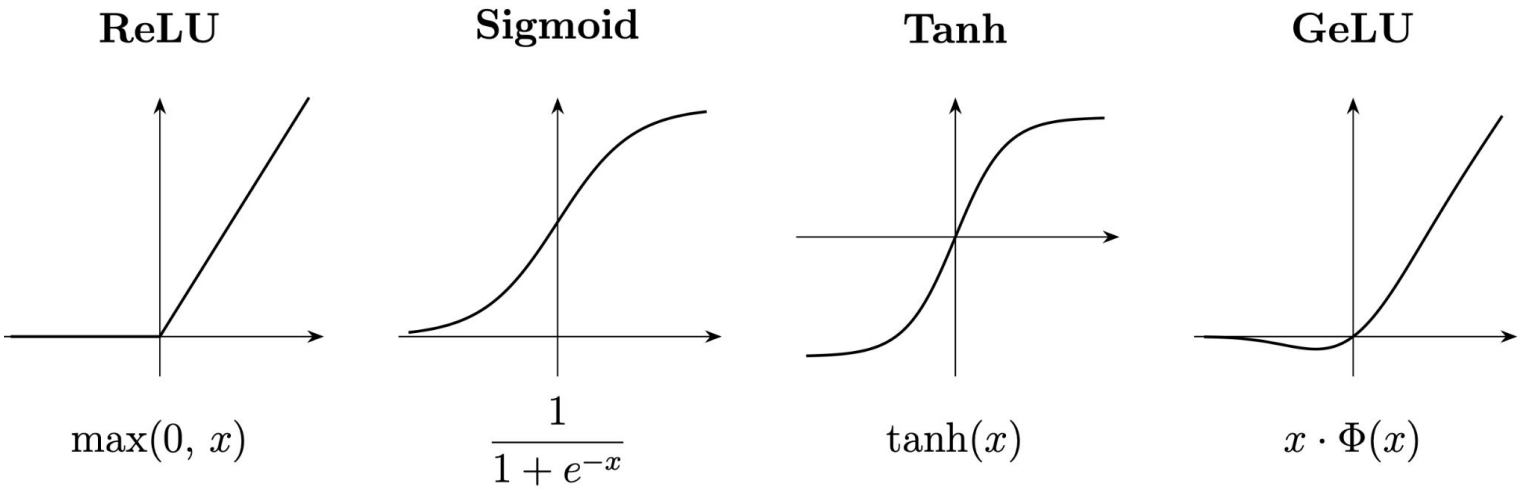
Issues with the original perceptron

- Practical problems are typically not linearly separable and require deep models
- The activation function (threshold) is not continuous
- The 0-1 error function is not continuous

- Use continuous and differentiable functions
  - To know if we are going the right way with some small change

Whiteboard: non-continous activation and error functions

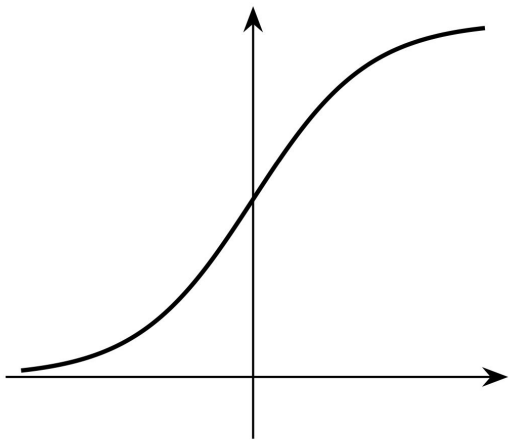
Smooth activation functions



- Differentiable almost everywhere
  - “Subderivatives” at non-differentiable corners (interval [a, b] from one-sided limits)

Sigmoid for probabilities

Sigmoid



$$\frac{1}{1+e^{-x}}$$

Sigmoid can model P(y=1|x)

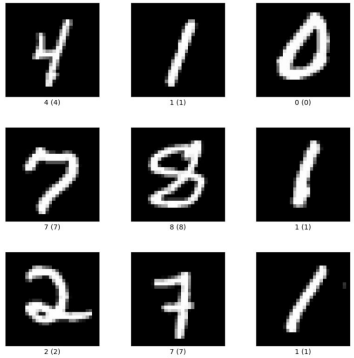
Whiteboard: sigmoid for P(y=1|x) in 1D

Softmax

A special activation that is applied to multiple neurons

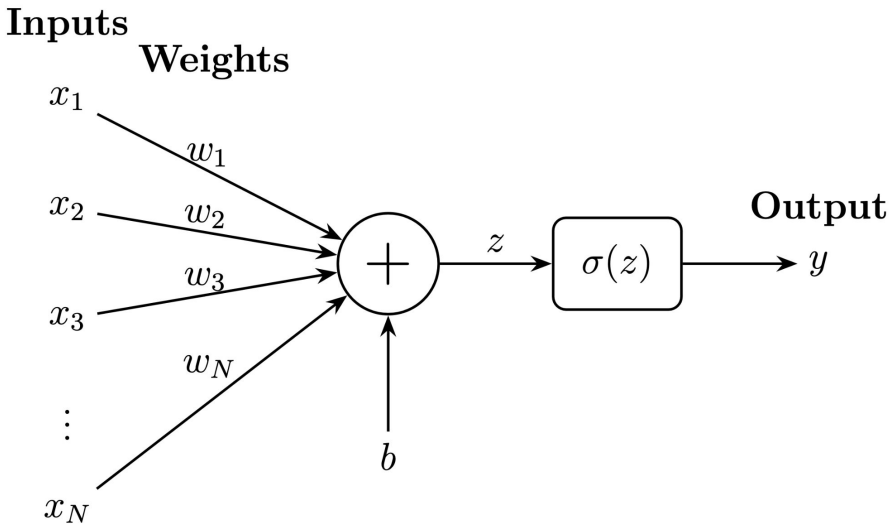
$$y_i = \mathrm{Softmax}(z) = \frac{\exp(z_i)}{\sum_j \exp(z_j)} \qquad \sum_i y_i = 1$$

Model probability P(y=i | x) in multi-class classification settings



Equivalent to softmax in binary classification

Differentiable model



$$\frac{dy}{dz} = \sigma'(z)$$

$$\frac{dy}{dw_i} = \frac{dy}{dz}\frac{dz}{dw_i} = \sigma'(z)x_i$$

$$\frac{dy}{dx_i} = \frac{dy}{dz}\frac{dz}{dx_i} = \sigma'(z)w_i$$

$$\frac{dy}{db} = \frac{dy}{dz}\frac{dz}{db} = \sigma'(z)$$

- y is differentiable with both inputs and parameters
- The derivatives tell us how the output changes for small changes in input or parameters

Differentiable error function

$$\text{empirical_error} = \sum_{i=1}^{n} \text{error}(y_i, g_i)$$

On i-th example: Output Ground-truth

- The error function should be differentiable
- Typically called “*loss*”

Loss functions

L2 divergence (squared Euclidean distance) for output vectors:

$$\mathrm{Loss}(y_i, g_i) = \frac{1}{2}\|y_i - g_i\|^2$$

Kullback Leibler (KL) divergence:

$$\mathrm{KL}(y_i, g_i) = \sum_j g_{ij} \log \frac{g_{ij}}{y_{ij}} = \sum_j g_{ij} \log g_{ij} - \sum_j g_{ij} \log y_{ij}$$

- Both reaches the minimum when y=g
- KL rises much faster when y is away from g

Whiteboard: derivatives and plot (binary)

Loss functions

- Multi-class classification:
  - One hot vector for g: [0 0 ... 1 ... 0 0 0]
  - g_c = 1 means the ground-truth class is c

- KL divergence: $\mathrm{Loss}(y_i, g_i) = -\log y_{ic_i}$

KL v.s. cross-entropy

KL divergence: $\mathrm{KL}(y_i, g_i) = \sum_j g_{ij} \log \frac{g_{ij}}{y_{ij}} = \sum_j g_{ij} \log g_{ij} - \sum_j g_{ij} \log y_{ij}$

How much information is lost when we use y to approximate g

Cross entropy: $H(y_i, g_i) = -\sum_j g_{ij} \log y_{ij} \quad H(y_i, g_i) = \mathrm{KL}(y_i, g_i) + H(g_i, g_i)$

How many bits (in expectation) are needed to identify an event from y if we use a coding scheme optimized for g

KL and cross entropy are equivalent when g is one-hot

Usually a cross-entropy loss is used for classification problems

Gradient

Partial derivatives:

$$df(X) = \nabla_X f(X) dX$$

$$\nabla_X f(X) = \begin{bmatrix} \frac{\partial f(X)}{\partial x_1} & \frac{\partial f(X)}{\partial x_2} & \cdots & \frac{\partial f(X)}{\partial x_d} \end{bmatrix}$$

Gradient:

$$\nabla_X f(X)^\top$$

Change in f(X) for any tiny variation in each x_i

f(x) changes the fastest when dX aligns with the gradient

Gradient descent

- Repeated update weights using **gradients w.r.t. weights**:

$$\mathbf{W}^{(t+1)} \leftarrow \mathbf{W}^{(t)} - \alpha \nabla_{\mathbf{W}^{(t)}} f(\mathbf{W}^{(t)})$$

- α is the step size (learning rate)
  - Too large: diverge
  - Too small: slow convergence

Whiteboard: demo

Gradient descent

- Successive approximation view:

  Form an approximation function at each iteration

$$f(\mathbf{W}^{(t)} + \mathbf{d}) \approx h(\mathbf{d}) := f(\mathbf{W}^{(t)}) + \nabla_{\mathbf{W}^{(t)}} f(\mathbf{W}^{(t)})^\top d + \frac{1}{2\alpha}\|\mathbf{d}\|^2$$

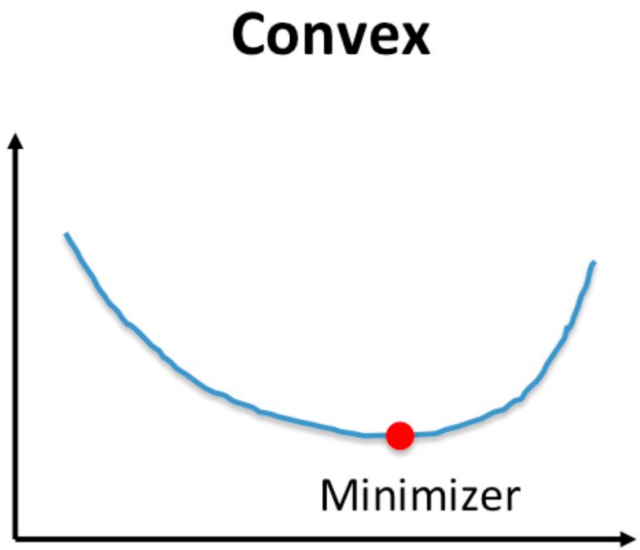
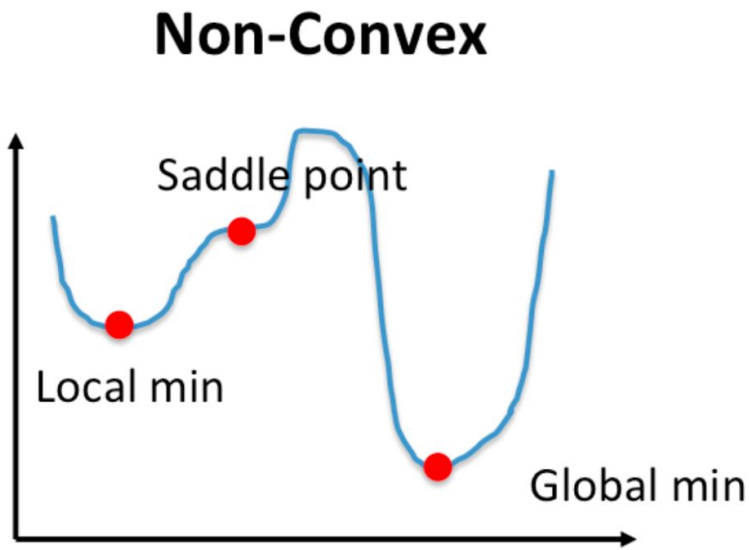
(regularizes step size)

- Update rule: $\mathbf{W}^{(t+1)} \leftarrow \mathbf{W}^{(t)} + \mathbf{d}^*$ $\quad \mathbf{d}^* = \arg\min_{\mathbf{d}} h(\mathbf{d})$

$$\nabla_{\mathbf{d}} g(\mathbf{d}^*) = 0 \Rightarrow \nabla_{\mathbf{W}^{(t)}} f(\mathbf{W}^{(t)}) + \frac{1}{\alpha}\mathbf{d}^* = 0 \qquad \Rightarrow \mathbf{d}^* = -\alpha \nabla_{\mathbf{W}^{(t)}} f(\mathbf{W}^{(t)})$$

(with sufficiently small α
and h is an upper bound of f)

Whiteboard: illustration (update rule with quadratic approximation)

Gradient descent

For non-convex functions (e.g., neural networks), optimization typically only converges to stationary points (not global minimum)